OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh | Register No: 192525458

GitHub Repository:

https://github.com/sharvesh-SR/CSA0421/tree/main/CSA-04%20Assignment%201/task1

TASK 1 – Evaluate the Requirements of Each Computing Environment (4 Marks)

Evaluate what each part of the hospital needs from its operating system.

Discuss:

• Response-time, reliability, throughput, battery and security requirements of each device.

• Which requirements conflict (for example: fast response and many users on one machine).

• Which requirements are critical to patient safety.

• How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment.

Rank the requirements of each environment and justify the ranking.

Alternative Concept and Complexity Analysis: If the whole hospital used a single general-purpose operating system on every device, evaluate worst-case alarm response time, memory needed on each device and administration effort.

Evidence Expected: A ranked requirement table for every environment with justification.

ANSWER

1.1 Ranked requirements

Environment	Priority ranking (1 = highest)	Reason for ranking
Bedside patient monitors	1. Deadline / response time; 2. reliability; 3. safety/security; 4. low memory/power; 5. throughput	An abnormal reading must trigger an alarm within 10 ms. A late alarm can be as dangerous as a missed alarm. The device must continue locally during network failure.
Nurses' handheld tablets	1. Security/privacy; 2. battery life; 3. reliability; 4. responsive UI; 5. throughput	Tablets are mobile and used throughout a shift. They need encrypted access to patient data, controlled screen locking and efficient power management.
Doctors' desktop workstations	1. Security; 2. interactive response; 3. reliability; 4. application compatibility; 5. throughput	Doctors need responsive access to charts, imaging viewers and clinical applications. Multi-user authentication and audit trails protect records.
Central patient-record database	1. Availability; 2. integrity/durability; 3. security; 4. throughput; 5. predictable recovery	The database must remain available during maintenance or hardware failure. Transactions, backups, replication and access controls are essential.
MRI imaging server	1. Throughput; 2. processor scalability; 3. memory bandwidth/capacity; 4. reliability; 5. security	MRI reconstruction is computationally intensive and benefits from parallel processors, large RAM and high-speed storage.
Partner-hospital link	1. Security; 2. availability; 3. integrity; 4. interoperability; 5. throughput	Records must be exchanged reliably across hospital boundaries using authenticated, encrypted and auditable communication. Network delays must not affect local alarms.

1.2 Conflicting requirements and patient-safety assessment

- Fast response conflicts with high system load: a general-purpose scheduler may run other tasks before an alarm task. A real-time scheduler with bounded execution and interrupt latency is needed.
- High throughput and many concurrent users can compete for CPU, memory and I/O. The database and imaging workloads should be isolated from monitor workloads.
- Strong security checks add processing overhead, but they cannot be removed from clinical data systems. Use efficient authentication, encryption acceleration and role-based access rather than disabling protection.
- Battery life conflicts with continuous wireless communication and frequent screen updates. Tablets should use power-aware scheduling, display sleep and controlled synchronisation.
- Patient-safety-critical functions are the monitor's sensor sampling, abnormal-value detection, alarm generation and local alarm output. They must work without depending on the cloud or hospital LAN.

1.3 Lessons from OS history

Historical development	Hospital application and evaluation
Batch systems	Batch processing is appropriate for non-urgent overnight backups, billing reports and archival jobs. It is unsuitable for bedside alarms because jobs do not guarantee immediate response.
Time-sharing systems	Time-sharing supports many doctors and nurses interacting with central applications. Fair sharing is useful for interactive work, but ordinary time-sharing alone cannot guarantee a 10 ms alarm deadline.
Personal/desktop systems	Desktop OSs provide graphical interfaces, drivers and broad application support for doctors' workstations. They need endpoint hardening and managed updates.
Mobile systems	Mobile OS features such as power management, secure boot, application sandboxing and wireless connectivity suit nurses' tablets. Battery and data-protection controls are essential.

1.4 Alternative: one general-purpose OS everywhere

If every device used a general-purpose OS, the design would be easier to administer because staff would maintain fewer OS families and common tools. However, a general-purpose scheduler does not normally guarantee a strict worst-case 10 ms alarm deadline. Under heavy CPU, storage or network load, scheduling delay can be variable and may be unbounded from the application's point of view. A hard real-time OS instead uses bounded interrupt and scheduling latency and priority-based pre-emption to make a deadline guarantee possible when the workload has been correctly analysed.

Memory: a full general-purpose OS usually requires more RAM and storage for services, drivers, background processes and a larger filesystem than a small embedded RTOS. This is wasteful on a constrained bedside monitor and can increase boot time and attack surface. Administration: a single OS reduces training and patching variety, but creates a common-mode failure and still requires device-specific configuration. Overall, the administration benefit does not outweigh the safety and resource disadvantages.

Complexity: for N devices, configuration and patching effort is broadly O(N) regardless of OS family; centralised management can reduce per-device manual effort but does not

change the need to manage each endpoint. A single OS does not improve alarm response-time complexity; the key issue is worst-case scheduling latency, not Big-O alone.